

Reviewer Pack — Minimal Rank of Hodge Structure over Tropical Varieties vs A...

Entry 5fe5db0b9a9e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 23:07:31 UTC

Minimal Rank of Hodge Structure over Tropical Varieties vs AC⁰ PARITY Depth

Entry ID: 5fe5db0b9a9e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 23:07:31 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: AC⁰ Circuit Complexity

Statement:

{‘concrete_statement’: ‘For every AC⁰ circuit C computing PARITY on n inputs with depth d, the minimal rank of the Hodge structure over its tropical variety is $\Theta(n^2 \log d)$.’, ‘falsifier_statement’: ‘There exists an AC⁰ circuit C computing PARITY on n inputs with depth d such that the minimal rank of the Hodge structure over its tropical variety is $o(n \log d)$.’}

Rationale (proposer’s reasoning):

{‘explanation_1’: ‘The Hodge theory of tropical varieties has shown potential in characterizing algebraic properties, and applying this to AC⁰ circuits could provide a new perspective on circuit complexity.’, ‘explanation_2’: ‘If the Hodge rank grows quadratically with the depth of an AC⁰ circuit computing PARITY, it would suggest that such circuits are more complex than currently understood.’}

Taxonomy category: AC⁰_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 05724738f510212b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal rank of the Hodge structure over the tropical variety of an AC⁰ PARITY circuit is $\Theta(n^2 \log d)$ if the mean minimal rank across all seeds is within 3 standard deviations from $n^2 \log d$ and at least 80% of seeds have a minimal rank within this range.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n, d):
        # Simplified AC^0 circuit generation (not actual AC^0)
        return [random.choice([0, 1]) for _ in range(d)]

    def tropical_variety(circuit):
        # Simplified tropical variety computation (not actual tropical variety)
        return sum(circuit) % 2

    def hodge_structure(variety):
        if isinstance(variety, int): # Handle the case where variety is an integer
            variety = [variety]
        return len(variety)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        for _ in range(5): # Test each n with 5 instances
            circuit = generate_ac0_circuit(n, n)
            variety = tropical_variety(circuit)
            hodge_rank = hodge_structure(variety)
            results.append(hodge_rank)

    mean_hodge_rank = sum(results) / len(results)
    std_deviation = math.sqrt(sum((x - mean_hodge_rank) ** 2 for x in results) / len(results))

    conjecture_holds = all(n**2 * math.log(d) <= mean_hodge_rank + 3 * std_deviation for n, d in z
```

```

counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Minimal Rank of Hodge Structure",
    "metric_value": mean_hodge_rank,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'Minimal Rank of Hodge Structure', 'metric_value': 1.0, 'instances_tested': 30}
TRIAL: {'metric_name': 'Minimal Rank of Hodge Structure', 'metric_value': 1.0, 'instances_tested': 30}
TRIAL: {'metric_name': 'Minimal Rank of Hodge Structure', 'metric_value': 1.0, 'instances_tested': 30}
TRIAL: {'metric_name': 'Minimal Rank of Hodge Structure', 'metric_value': 1.0, 'instances_tested': 30}
TRIAL: {'metric_name': 'Minimal Rank of Hodge Structure', 'metric_value': 1.0, 'instances_tested': 30}
TRIAL: {'metric_name': 'Minimal Rank of Hodge Structure', 'metric_value': 1.0, 'instances_tested': 30}
TRIAL: {'metric_name': 'Minimal Rank of Hodge Structure', 'metric_value': 1.0, 'instances_tested': 30}
TRIAL: {'metric_name': 'Minimal Rank of Hodge Structure', 'metric_value': 1.0, 'instances_tested': 30}
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2681536c.py", line 77, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2681536c.py", line 77, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    ~~~~~^

```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Review and debug the test code to ensure it can complete without crashing and produce the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11636
2	propose	ollama_remote	glm4:latest	0	0	10604
3	preregistration	ollama_remote	glm4:latest	0	0	6123
4	novelty	ollama_remote	glm4:latest	0	0	4849
5	novelty	ollama_remote	glm4:latest	0	0	5179
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45872
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10140
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8707
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8337
10	judge	ollama_remote	glm4:latest	0	0	11268

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122714 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5fe5db0b9a9e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5fe5db0b9a9e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5fe5db0b9a9e.tar.gz` (if generated)